ESTR1002 Fall 2025

Lab Exercise #7

Due: 5 Nov 2025

Separate your answer for each question in a different .c file. Compress all files into a .zip and submit it to Blackboard.

Question 1

Given a text paragraph and then a list of numbers, use the numbers in sequence as indices to grab the alphabets in the given paragraph to find out the secret words hidden in the paragraph.

Input: the input contains two parts.

- The first part is a single paragraph of text. The paragraph should contain only alphabets, punctuations, and spaces (no newlines). The paragraph should not exceed 200 characters.
- The second part is a list of numbers separated by whitespace. The maximum length of the list is 200.

Output: the secret words formed by the alphabets at the given indices in the paragraph. The output should be in lowercase.

The sample output is shown as follows (the underlined characters are the inputs from the user).

Sample Run #1:

```
Alpe
1 3 3 2 4
apple
```

Sample Run #2:

```
The quick brown fox jumps over the lazy dog
25 1 3 28 3 4 21 13 11 25
steve jobs
```

Question 2

Consider the following code:

```
#include <stdio.h>

int main(void)
{
    printf("%c\n", '日');
    printf("\101\n");
    return 0;
}
```

There is a Chinese character we wanted to print but failed.

Task: Fix the bug.

Question 3

Monte Carlo is a widely used way to get numerical results. In fact it is a kind of simulation based on random sampling. A toy example is to estimate the value of π.

Suppose we have a unit square and draw a circle with radius 0.5 in it. If we sample points in the square uniformly many many times, the ratio

(number of points hit the circle) : (total number of points)

should be close to the ratio

(area of the circle) : (area of the square)

which equals $\frac{\pi}{4}$. That means that we can estimate the value of π.

Task: Print the value of π estimated by Monte Carlo simulation.

Remarks: In C, there is a function called rand() which will give an integer in the range [0, RAND_MAX], where the function prototype and RAND_MAX are defined in stdlib.h

If you want sample a uniform distribution over [0,1) (with limited precision), a simple way is to compute

```
rand()/(RAND_MAX+1.)
```

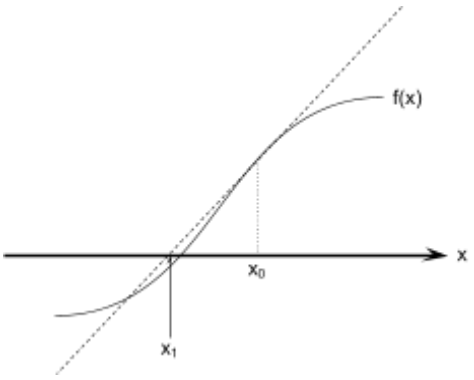
If you want to sample a uniform distribution over (0,1), then compute

```
(rand()+.5)/(RAND_MAX+1.)
```

Question 4

You will face different kinds of equations you need to solve in your engineering life. Unfortunately, you may not know how to solve all of them. As an engineer, you will use numerical methods to approximate the solution.

There are tons of numerical methods to find the root of an equation like the bisection method, secant method, false position method, etc. Today we are going to implement a famous method named Newton's method.



The idea is simple. Suppose we are going to find a real root of a continuous function $f(x)$, and we assume the derivative $f'(x)$ exists for all real x. Given an initial guess x_0, if it is not a root, we can find the tangent line at that point. The equation of the tangent is $y = f'(x_0)\,x + c$, and we can solve c by $c = f(x_0) - f'(x_0)x_0$. So we have $y = f'(x_0)(x - x_0) + f(x_0)$. The next guess x_1 is the x-intercept of this tangent line. That is $0 = f'(x_0)(x_1 - x_0) + f(x_0)$, which gives $x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$. We can repeat the procedure until we solve $f(x)$ or we get a close enough solution.

From the above you can see the method will fail under some conditions like $f'(x_n) = 0$ for some n. In fact you need to have an initial guess “close enough” to the root to ensure the method converges. But in practice you won’t know the range of the initial guesses which work. So we need to fix a maximum number of iterations, i.e., after a fixed number of steps we will stop.

Sometimes it may only improve a bit if you try more iterations, so we can stop the method if the change between two iterations is too small.

Task: Find all (approximated) roots of the cubic equation $ax^3 + bx^2 + cx + d = 0$, where the coefficients are all real and read from user input. Apply Newton's method to find a real root, and then do polynomial division to get a quadratic equation for solving the remaining (may be real or complex) roots.

(Beware the case when $a = 0$)